

数据结构（C++版）期末题型完全攻略

基于王红梅、胡明编著《数据结构(C++版)(第2版)》及历年真题编制 适用课程：数据结构、数据结构与算法

🔍 考试题型总览

题型	分值占比	难度层次	备考建议
单项选择题	20-30%	基础	概念辨析、复杂度分析
填空题	15-20%	基础→中等	公式、性质、概念
判断题	10-15%	基础	概念正确性判断
简答题/应用题	25-30%	中等→综合	画图、计算、序列推导
算法设计题	15-20%	综合	代码填空、完整算法

第1章 绪论

1.1 基本概念

必背知识

概念	定义
数据结构	相互之间存在一种或多种特定关系的数据元素的集合
数据元素	数据的基本单位，在计算机中通常作为一个整体进行考虑和处理
数据项	构成数据元素的不可分割的最小单位
逻辑结构	数据元素之间的逻辑关系，与存储无关
存储结构（物理结构）	数据元素及其关系在计算机内的存储方式

逻辑结构类型

线性结构	非线性结构
├─ 线性表	├─ 树
├─ 栈	├─ 图
├─ 队列	
└─ 串	

存储结构类型

类型	特点	优点	缺点
顺序存储	连续存储单元	可随机访问	需连续空间，插入删除不便
链式存储	指针链接	动态分配，插入删除方便	占用额外空间，不可随机访问
索引存储	建立索引表	检索快	索引表占空间
散列存储	散列函数	检索快	冲突处理

1.2 算法与算法分析

必背知识

算法的5个重要特性

- 1. 有输入：可有可无
- 2. 有输出：至少有一个输出
- 3. 确定性：每条指令无歧义
- 4. 可行性：基本操作可执行
- 5. 有穷性：执行有限步后终止

时间复杂度

定义：算法执行时间与问题规模n的函数T(n)

常见复杂度等级（从低到高）：

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n)$$

计算规则：

- 加法规则： $T(n) = T_1(n) + T_2(n) = O(\max(f(n), g(n)))$
- 乘法规则： $T(n) = T_1(n) \times T_2(n) = O(f(n) \times g(n))$

空间复杂度

定义：算法执行所需的存储空间与问题规模的函数S(n)

类型	说明
固定部分	程序代码、变量等，与n无关
可变部分	动态分配的空间、递归栈等，与n有关

1.3 题型攻略

题型一：时间复杂度计算

识别特征

- 给定代码段，求时间复杂度
- 给定问题规模，求基本操作执行次数

解法步骤

步骤1：找出语句频度最大的语句（基本语句）
步骤2：分析该语句的执行次数与n的关系
步骤3：使用渐进表示法O(...)

典型例题

例1：

```
for(i=1; i<=n; i++)
    for(j=1; j<=n; j++)
        x++;
```

解答：基本语句 x++ ，执行次数 = $n \times n = n^2 \rightarrow O(n^2)$

例2：

```
i=1;
while(i<=n)
    i=i*2;
```

解答：设执行t次， $2^t \leq n \rightarrow t \leq \log_2 n \rightarrow O(\log_2 n)$

例3（2019-2020真题）：

```
for(k=1; k<=n; k=k*2)
    for(j=1; j<=n; j++)
        x++;
```

解答：外层 $\log_2 n$ 次，内层 n 次 $\rightarrow O(n \log_2 n)$

题型二：数据结构基本概念判断

考法与解法

考法	识别特征	解法
存储结构与逻辑结构	"采用...存储"描述	顺序/链式→存储结构，树/图→逻辑结构
算法的特性	考察5个特性	有穷性、确定性等关键词
时间复杂度计算	给代码求复杂度	分析循环层数
空间复杂度计算	给代码求空间	找动态分配/递归深度

第2章 线性表

2.1 线性表的顺序存储

必背知识

顺序表结构体定义

```
struct SqList {
    ElemType *elem;    // 存储空间基地址
    int length;        // 当前长度
    int listsize;      // 分配的存储容量
};
```

顺序表特点

- 地址连续: $LOC(a_i) + (i-1) \times k$
- 随机存取: 时间复杂度 $O(1)$
- 存储密度: $= 1$ (100%利用)

基本操作时间复杂度

操作	时间复杂度
存取第i个元素	$O(1)$
查找	$O(n)$
插入	$O(n)$
删除	$O(n)$

2.2 线性表的链式存储

必背知识

单链表结构体定义

```
struct LNode {
    ElemType data;
    struct LNode *next;
};
typedef struct LNode *LinkList;
```

单链表特点

- 无需连续空间
- 通过指针链接
- 头结点: 第一个结点前的人工结点 (不含数据)

循环链表 vs 双链表

类型	特点	判空条件
单链表	只含next指针	head->next == NULL
循环单链表	尾结点next指向头结点	head->next == head
双链表	含prior和next指针	-
循环双链表	头结点的prior指向尾结点	head->prior == head

单链表基本操作时间复杂度

操作	时间复杂度	说明
建立（头插法）	O(n)	每次头插
建立（尾插法）	O(n)	每次尾插
按序号查找	O(n)	最坏情况
按值查找	O(n)	最坏情况
插入	O(n)	定位+插入
删除	O(n)	定位+删除

2.3 栈和队列

必背知识

栈（后进先出 LIFO）

顺序栈结构体定义：

```
struct SqStack {
    ElemType *base; // 栈底指针
    ElemType *top;  // 栈顶指针
    int stacksize;  // 栈容量
};
```

栈的基本操作：

操作	说明
InitStack(&S)	初始化空栈
Push(&S, e)	压栈
Pop(&S, &e)	弹栈
GetTop(S, &e)	取栈顶
StackEmpty(S)	判空

栈空条件： `S.top == S.base` 栈满条件： `S.top - S.base == S.stacksize`

队列（先进先出 FIFO）

循环队列结构体定义：

```
struct SqQueue {
    ElemType *base; // 存储空间基地址
    int front;      // 头指针
    int rear;       // 尾指针
};
```

循环队列关键公式：

操作	公式
队空条件	<code>Q.front == Q.rear</code>
队满条件	<code>(Q.rear + 1) % MAXSIZE == Q.front</code>
队列长度	<code>(Q.rear - Q.front + MAXSIZE) % MAXSIZE</code>
入队	<code>Q.rear = (Q.rear + 1) % MAXSIZE</code>
出队	<code>Q.front = (Q.front + 1) % MAXSIZE</code>

2.4 题型攻略

题型一：链表基本操作

单链表删除结点（第i个结点后）

```
p = GetElem(L, i);    // 找到第i个结点
q = p->next;          // q指向第i+1个结点
p->next = q->next;     // 从链上摘下q
free(q);              // 释放q
```

单链表插入结点（第i个结点后）

```
p = GetElem(L, i);    // 找到第i个结点
s = new LNode;        // 创建新结点
s->data = e;
s->next = p->next;     // 新结点指向后继
p->next = s;          // 前驱指向新结点
```

链表逆置（头插法）

```
void ReverseList(LinkList &L) {
    LinkList p = L->next;    // p指向第一个数据结点
    L->next = NULL;         // 头结点next置空
    while(p != NULL) {
        LinkList q = p->next; // 保存后继
        p->next = L->next;    // 头插
        L->next = p;
        p = q;               // 移动p
    }
}
```

题型二：栈的应用——出栈序列判断

识别特征

- 给出进栈序列和出栈序列
- 判断是否可能/求栈容量

解法步骤

```
步骤1：理解栈的LIFO特性
步骤2：模拟入栈出栈过程
步骤3：判断出栈序列合法性
```

典型例题（2018-2019真题）

题目：进栈次序为A,B,C,D,E，以下出栈序列不可能的是？

- A. A,B,C,D,E
- B. B,C,D,E,A
- C. E,A,B,C,D
- D. E,D,C,B,A

解答：C不可能，因为E先出说明A,B,C,D都在栈中，出栈顺序必须是D,C,B,A

题型三：循环队列元素个数

识别特征

- 循环队列数组下标范围1~n
- 头尾指针分别为f和r

解法公式

```
元素个数 = ( r - f + n ) % n
```

第3章 字符串和多维数组

3.1 字符串

必背知识

概念	说明
串长	串中字符的个数
空串	长度为零的串
空格串	仅含空格的串
子串	串中任意连续字符组成的序列
主串	包含子串的串

串的存储结构

```
struct HString {
    char *ch;    // 字符数组
    int length;  // 串长
};
```

3.2 模式匹配（重点）

必背知识——KMP算法

Next数组计算

核心思想：当匹配失败时，利用已经部分匹配的信息，不回溯主串指针

Next[j]含义：模式串第j个字符匹配失败时，应跳转到的位置

计算公式：

```
Next[1] = 0
Next[2] = 1
Next[j] = k ( 当 P[1..k-1] = P[j-k+1..j-1] 时, k为最长相等前后缀长度+1 )
```

Nextval数组（Next的改进）

当P[j] = P[Next[j]]时，Nextval[j] = Nextval[Next[j]]

标准化做题步骤

```
步骤1：写出模式串P
步骤2：按位置标号1到m
步骤3：计算每个位置的Next值
步骤4：若需要，计算Nextval值
```

典型例题

例：求模式串 "abaabcac" 的next数组

j	1	2	3	4	5	6	7	8
P	a	b	a	a	b	c	a	c
Next	0	1	1	2	2	3	1	2
Nextval	0	1	0	2	1	3	0	2

3.3 多维数组与矩阵压缩

必背知识

数组地址计算

按行优先（C/C++/Pascal）：

$$LOC(i,j) = LOC(0,0) + (i \times n + j) \times k$$

按列优先（Fortran）：

$$LOC(i,j) = LOC(0,0) + (j \times m + i) \times k$$

特殊矩阵压缩存储

矩阵类型	压缩存储大小	存储方式
对称矩阵	$n(n+1)/2$	只存下三角（含对角线）
三角矩阵	$n(n+1)/2$	只存下三角+常数
对角矩阵	$3n-2$	只存三条对角线

第4章 树和二叉树

4.1 二叉树基本概念

必背知识

二叉树5种基本形态



二叉树性质

性质	内容
性质1	在二叉树第 <i>i</i> 层上至多有 $2^{(i-1)}$ 个结点
性质2	深度为 <i>k</i> 的二叉树至多有 $2^k - 1$ 个结点
性质3	终端结点数 = 度为2的结点数 + 1
性质4	完全二叉树深度 = $\lfloor \log_2 n \rfloor + 1$

完全二叉树性质

- 结点*i*的左孩子： $2i$ （若 $2i \leq n$ ）

- 结点*i*的右孩子: $2i+1$ (若 $2i+1 \leq n$)
- 结点*i*的父节点: $\lfloor i/2 \rfloor$

4.2 二叉树遍历（核心）

必背知识

遍历序列关系

遍历方式	序列特点	根的位置
先序（DLR）	根左右	第一个
中序（LDR）	左根右	中间
后序（LRD）	左右根	最后一个
层序	按层从上到下	按层访问

由遍历序列重建二叉树

关键：必须知道中序序列 + 任意一种其他遍历序列

做题步骤：

1. 从先序/后序确定根

2. 在中序中划分左右子树

3. 递归处理左右子树

4.3 题型攻略

题型一：二叉树遍历代码

先序遍历递归算法

```
void PreOrder(BiTree T) {
    if(T != NULL) {
        Visit(T->data);    // 访问根
        PreOrder(T->lchild); // 遍历左子树
        PreOrder(T->rchild); // 遍历右子树
    }
}
```

中序遍历递归算法

```
void InOrder(BiTree T) {
    if(T != NULL) {
        InOrder(T->lchild);
        Visit(T->data);
        InOrder(T->rchild);
    }
}
```

后序遍历递归算法

```
void PostOrder(BiTree T) {
    if(T != NULL) {
        PostOrder(T->lchild);
        PostOrder(T->rchild);
        Visit(T->data);
    }
}
```

二叉树结构体定义


```
struct BiTNode {
    ElemType data;
    struct BiTNode *lchild, *rchild;
};
typedef struct BiTNode *BiTree;
```

题型二：给定中序和后序求先序（2018-2019真题）

已知：中序 cbedahgjff，后序 cedbhjigfa 求：先序序列

解题步骤：

1. 从后序最后一个确定根：a
2. 在中序中划分：
 - cbed 在左子树
 - hgijf 在右子树
3. 后序 cedb → 左子树根为 b
后序 hjgif → 右子树根为 f
4. 递归继续...
5. 最终得到先序：abcdefghij

题型三：哈夫曼树与哈夫曼编码

哈夫曼树构造步骤

- 步骤1：把所有权值放入集合S
步骤2：选取S中最小的两个权值w1、w2
步骤3：创建新结点，权值=w1+w2，作为它们的父结点
步骤4：从S中删除w1、w2，加入新结点权值
步骤5：重复步骤2-4，直到S只剩一个权值

注意：左子树根权值 $\leq$ 右子树根权值

哈夫曼编码特点

- 前缀编码：任一字符编码都不是另一字符编码的前缀
- 最优编码：WPL最小
- 左分支为0，右分支为1

代码模板

```
typedef struct {
    int weight;
    int parent, lchild, rchild;
} HTNode, *HuffmanTree;

void HuffmanCoding(HuffmanTree &HT, char **HC, int *w, int n) {
    if(n <= 1) return;
    int m = 2 * n - 1;
    HT = new HTNode[m + 1];

    // 初始化
    for(int i = 1; i <= m; i++) {
        HT[i].parent = HT[i].lchild = HT[i].rchild = 0;
    }
    for(int i = 1; i <= n; i++)
        HT[i].weight = w[i-1];

    // 构建哈夫曼树
    for(int i = n + 1; i <= m; i++) {
        int s1, s2;
        Select(HT, i - 1, s1, s2); // 选最小两结点
        HT[s1].parent = HT[s2].parent = i;
        HT[i].lchild = s1;
        HT[i].rchild = s2;
        HT[i].weight = HT[s1].weight + HT[s2].weight;
    }

    // 从叶子到根逆向求编码
    char *cd = new char[n];
    cd[n-1] = '\0';
    for(int i = 1; i <= n; i++) {
        int start = n - 1;
        for(int c = i, f = HT[i].parent; f != 0; c = f, f = HT[f].parent) {
            if(HT[f].lchild == c) cd[--start] = '0';
            else cd[--start] = '1';
        }
        HC[i] = new char[n - start];
        strcpy(HC[i], &cd[start]);
    }
    delete[] cd;
}
```

4.4 平衡二叉树（AVL）

必背知识

类型	情况	调整方法
LL型	左左	单右旋
RR型	右右	单左旋
LR型	左右	先左旋后右旋
RL型	右左	先右旋后左旋

平衡因子

- BF = 左子树深度 - 右子树深度
- AVL树: $|BF| \leq 1$

第5章 图

5.1 图的存储结构

必背知识

邻接矩阵

```
#define MAX_VERTEX_NUM 20
typedef struct {
    VertexType vexs[MAX_VERTEX_NUM]; // 顶点信息
    EdgeType arc[MAX_VERTEX_NUM][MAX_VERTEX_NUM]; // 边信息
    int vexnum, arcnum; // 顶点数、边数
} MGraph;
```

特点:

- 无向图对称矩阵
- 有向图不一定对称
- 空间复杂度: $O(n^2)$

邻接表

```
struct ArcNode {
    int adjvex; // 邻接点位置
    struct ArcNode *nextarc;
};

typedef struct {
    VertexType data;
    ArcNode *firstarc;
} VNode, AdjList[MAX_VERTEX_NUM];

struct ALGraph {
    AdjList vertices;
    int vexnum, arcnum;
};
```

特点:

- 空间复杂度: $O(n+e)$
- 适合稀疏图

5.2 图的遍历

必背知识

深度优先遍历 (DFS)

- 类似先序遍历
- 使用栈/递归
- 时间复杂度: $O(n+e)$

广度优先遍历 (BFS)

- 类似层序遍历
- 使用队列
- 时间复杂度: $O(n+e)$

代码模板

```

// 邻接表的DFS
bool visited[MAX_VERTEX_NUM];

void DFS(ALGraph G, int v) {
    visited[v] = true;
    Visit(G.vertices[v].data);
    ArcNode *p = G.vertices[v].firstarc;
    while(p != NULL) {
        if(!visited[p->adjvex])
            DFS(G, p->adjvex);
        p = p->nextarc;
    }
}

// 邻接表的BFS
void BFS(ALGraph G, int v) {
    queue<int> Q;
    visited[v] = true;
    Visit(G.vertices[v].data);
    Q.push(v);
    while(!Q.empty()) {
        int u = Q.front();
        Q.pop();
        ArcNode *p = G.vertices[u].firstarc;
        while(p != NULL) {
            if(!visited[p->adjvex]) {
                visited[p->adjvex] = true;
                Visit(G.vertices[p->adjvex].data);
                Q.push(p->adjvex);
            }
            p = p->nextarc;
        }
    }
}

```

5.3 最小生成树

必背知识

Prim算法（普里姆算法）

基本思想：从某顶点出发，逐步扩展，每次选择权值最小的边

时间复杂度： $O(n^2)$

```

void Prim(MGraph G, int v0) {
    int lowcost[MAX_VERTEX_NUM];
    int adjvex[MAX_VERTEX_NUM];

    // 初始化
    for(int i = 0; i < G.vexnum; i++) {
        lowcost[i] = G.arc[v0][i];
        adjvex[i] = v0;
    }
    lowcost[v0] = 0; // 标记已加入

    for(int i = 1; i < G.vexnum; i++) {
        int k = MinEdge(lowcost, G.vexnum);
        lowcost[k] = 0; // 加入生成树
        // 更新lowcost和adjvex
        for(int j = 0; j < G.vexnum; j++) {
            if(G.arc[k][j] < lowcost[j]) {
                lowcost[j] = G.arc[k][j];
                adjvex[j] = k;
            }
        }
    }
}

```

Kruskal算法（克鲁斯卡尔算法）

基本思想：按权值从小到大选边，不形成环

时间复杂度： $O(e \log e)$

核心：使用并查集判断是否成环

贪心选择：每次选权值最小且不成环的边

5.4 最短路径

必背知识

Dijkstra算法（单源最短路径）

适用：非负权图

时间复杂度： $O(n^2)$

核心： $\text{dist}[i] = \min(\text{当前dist}[i], \text{dist}[k] + \text{arc}[k][i])$

```
void Dijkstra(MGraph G, int v0) {
    int dist[MAX_VERTEX_NUM];
    int S[MAX_VERTEX_NUM];

    // 初始化
    for(int i = 0; i < G.vexnum; i++) {
        dist[i] = G.arc[v0][i];
        S[i] = 0;
    }
    S[v0] = 1;

    for(int i = 1; i < G.vexnum; i++) {
        int u = 0, min = INFINITY;
        // 找最小dist
        for(int j = 0; j < G.vexnum; j++) {
            if(!S[j] && dist[j] < min) {
                min = dist[j];
                u = j;
            }
        }
        S[u] = 1;
        // 更新dist
        for(int w = 0; w < G.vexnum; w++) {
            if(!S[w] && dist[u] + G.arc[u][w] < dist[w])
                dist[w] = dist[u] + G.arc[u][w];
        }
    }
}
```

5.5 拓扑排序

必背知识

AOV网：用顶点表示活动的有向无环图

拓扑排序：将AOV网中所有顶点排成线性序列

特点：

- 若存在拓扑序列，图无环
- 若图无环，必存在拓扑序列

算法步骤：

1. 计算所有顶点入度
2. 将入度为0的顶点入栈
3. 栈非空时，弹出顶点，输出
4. 将其邻接点的入度-1
5. 若邻接点入度变为0，入栈

第6章 查找

6.1 顺序查找

必背知识

时间复杂度: $O(n)$

监视哨方法:

```
int Search_Seq(SSTable ST, KeyType key) {
    ST.R[0].key = key; // 监视哨
    int i = ST.length;
    while(ST.R[i].key != key) i--;
    return i;
}
```

优点: 查找成功时未比较元素 缺点: 需要0号单元

6.2 折半查找（二分查找）

必背知识

前提条件:

1. 顺序存储结构
2. 有序序列

判定树: 折半查找的过程可用二叉树描述

时间复杂度: $O(\log_2 n)$

```
int Search_Bin(SSTable ST, KeyType key) {
    int low = 1, high = ST.length;
    while(low <= high) {
        int mid = (low + high) / 2;
        if(ST.R[mid].key == key)
            return mid;
        else if(ST.R[mid].key > key)
            high = mid - 1;
        else
            low = mid + 1;
    }
    return 0;
}
```

折半查找判定树

性质:

- 查找次数 = 结点所在层数
- $ASL \approx \log_2(n+1) - 1$

6.3 二叉排序树（BST）

必背知识

定义:

- 左子树 < 根 < 右子树
- 左右子树均为BST

查找:

```
BSTNode* Search_BST(BSTree T, KeyType key) {
    if(T == NULL || T->data == key)
        return T;
    if(key < T->data)
        return Search_BST(T->lchild, key);
    else
        return Search_BST(T->rchild, key);
}
```

时间复杂度:

- 平均: $O(\log_2 n)$
- 最坏（退化为链表）: $O(n)$

6.4 散列查找（哈希表）

必背知识

哈希函数

方法	公式	适用场景
直接定址法	$H(key) = a \times key + b$	key分布均匀
除留余数法	$H(key) = key \% p$	通用
数字分析法	取key中分布均匀的位	key已知
平方取中法	取 key^2 中间几位	key分布均匀

冲突处理

开放地址法：

方法	增量序列
线性探测	$d_i = 1, 2, 3, \dots, m-1$
二次探测	$d_i = 1^2, -1^2, 2^2, -2^2, \dots$
伪随机探测	$d_i = \text{伪随机数序列}$

拉链法：

- 将同义词存储在同一个链表中
- 适合插入删除频繁的场景

装填因子

$\alpha = n / m$ （n为元素数，m为表长）
 α 越大，冲突越多

平均查找长度

ASL成功 = 各元素查找次数之和 / 元素个数
ASL失败 = 各位置查找次数之和 / 地址个数

第7章 排序

7.1 排序算法总览

必背知识——复杂度总结

排序算法	最好	平均	最坏	空间	稳定性
直接插入	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	
希尔排序	$O(n^{1.3})$	$O(n^{1.5})$	$O(n^2)$	$O(1)$	
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	
快速排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n^2)$	$O(\log_2 n)$	
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	
堆排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(1)$	
归并排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n)$	
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	

7.2 插入排序

直接插入排序

```
void InsertSort(SqList &L) {
    for(int i = 2; i <= L.length; i++) {
        if(L.r[i].key < L.r[i-1].key) {
            L.r[0] = L.r[i]; // 监视哨
            int j = i - 1;
            while(L.r[0].key < L.r[j].key) {
                L.r[j+1] = L.r[j];
                j--;
            }
            L.r[j+1] = L.r[0];
        }
    }
}
```

希尔排序

```
void ShellSort(SqList &L, int dlta[], int t) {
    for(int k = 0; k < t; k++) {
        int dk = dlta[k]; // 增量
        for(int i = dk + 1; i <= L.length; i++) {
            if(L.r[i].key < L.r[i-dk].key) {
                L.r[0] = L.r[i];
                int j = i - dk;
                while(j > 0 && L.r[0].key < L.r[j].key) {
                    L.r[j+dk] = L.r[j];
                    j -= dk;
                }
                L.r[j+dk] = L.r[0];
            }
        }
    }
}
```

7.3 交换排序

冒泡排序

```
void BubbleSort(SqList &L) {
    bool exchange;
    for(int i = 1; i < L.length; i++) {
        exchange = false;
        for(int j = 1; j <= L.length - i; j++) {
            if(L.r[j].key > L.r[j+1].key) {
                Swap(L.r[j], L.r[j+1]);
                exchange = true;
            }
        }
        if(!exchange) return; // 已排好序
    }
}
```

快速排序（重点）


```

int Partition(SqlList &L, int low, int high) {
    KeyType pivotkey = L.r[low].key; // 枢轴
    while(low < high) {
        while(low < high && L.r[high].key >= pivotkey)
            high--;
        L.r[low] = L.r[high];
        while(low < high && L.r[low].key <= pivotkey)
            low++;
        L.r[high] = L.r[low];
    }
    L.r[low] = L.r[0];
    return low;
}

void QSort(SqlList &L, int low, int high) {
    if(low < high) {
        int pivotloc = Partition(L, low, high);
        QSort(L, low, pivotloc - 1);
        QSort(L, pivotloc + 1, high);
    }
}

void QuickSort(SqlList &L) {
    QSort(L, 1, L.length);
}

```

第一趟划分：以第一个元素为枢轴，将序列分为两部分

7.4 选择排序

简单选择排序

```

void SelectSort(SqlList &L) {
    for(int i = 1; i < L.length; i++) {
        int k = i;
        for(int j = i + 1; j <= L.length; j++) {
            if(L.r[j].key < L.r[k].key)
                k = j;
        }
        if(k != i)
            Swap(L.r[i], L.r[k]);
    }
}

```

堆排序（重点）

```

void HeapAdjust(HeapType &H, int s, int m) {
    // 调整H.r[s]为大顶堆
    RedType rc = H.r[s];
    for(int j = 2 * s; j <= m; j *= 2) {
        if(j < m && H.r[j].key < H.r[j+1].key)
            j++; // j指向较大的孩子
        if(rc.key >= H.r[j].key)
            break;
        H.r[s] = H.r[j];
        s = j;
    }
    H.r[s] = rc;
}

void HeapSort(HeapType &H) {
    // 建立初始堆
    for(int i = H.length / 2; i > 0; i--)
        HeapAdjust(H, i, H.length);

    // 堆排序
    RedType t;
    for(int i = H.length; i > 1; i--) {
        t = H.r[1];
        H.r[1] = H.r[i];
        H.r[i] = t;
        HeapAdjust(H, 1, i - 1);
    }
}

```

7.5 归并排序

```
void Merge(int SR[], int TR[], int i, int m, int n) {
    // 将有序SR[i..m]和SR[m+1..n]归并为TR[i..n]
    int j, k;
    for(j = m + 1, k = i; i <= m && j <= n; k++) {
        if(SR[i] <= SR[j])
            TR[k] = SR[i++];
        else
            TR[k] = SR[j++];
    }
    while(i <= m) TR[k++] = SR[i++];
    while(j <= n) TR[k++] = SR[j++];
}

void MSort(int SR[], int TR1[], int s, int t) {
    int TR2[MAXSIZE];
    if(s == t)
        TR1[s] = SR[s];
    else {
        int m = (s + t) / 2;
        MSort(SR, TR2, s, m);
        MSort(SR, TR2, m + 1, t);
        Merge(TR2, TR1, s, m, t);
    }
}
```

题型实战——历年真题精选

题型一：快速排序第一趟（必考）

真题（2019-2020）：序列(24, 19, 32, 43, 38, 6, 13, 22)，以第一个记录为枢轴，第一趟排序结果？

答案：22 19 13 6 24 38 43 32

解题要点：

1. 基准数一直不变，直到找到最终位置
2. 先从后往前找小于基准的，再从前往后找大于基准的
3. 交替进行直到low==high

题型二：哈希表构造（必考）

真题（2018-2019）： $H(\text{key}) = \text{key} \% 13$ ，线性探测再散列 插入：19, 1, 23, 14, 55, 2, 84, 27, 68, 11, 10, 78

解题步骤：

```
H(19) = 19 % 13 = 6 → 地址6
H(1) = 1 % 13 = 1 → 地址1
H(23) = 23 % 13 = 10 → 地址10
H(14) = 14 % 13 = 1 → 地址1冲突，探查2 → 地址2
H(55) = 55 % 13 = 3 → 地址3
H(2) = 2 % 13 = 2 → 地址2冲突，探查3,4 → 地址4
H(84) = 84 % 13 = 6 → 地址6冲突，探查7,8,9 → 地址8
H(27) = 27 % 13 = 1 → 地址1冲突... → 地址5
...
```

ASL计算：

- 每个元素查找次数之和 / 元素个数

题型三：完全二叉树判断（算法设计）

真题（2018-2019）：设计判断二叉树是否完全二叉树的算法

解题思路：

层序遍历二叉树：

1. 若遇到空结点，之后所有结点必须都为空
2. 若遇到非空结点，其左右子结点都应入队

代码模板：

```
int IsCompleteTree(BTree T) {
    if(T == NULL) return 1;
    Queue Q;
    BTree p;
    InitQueue(Q);
    EnQueue(Q, T);

    while(!QueueEmpty(Q)) {
        DeQueue(Q, p);
        if(p == NULL) break;
        EnQueue(Q, p->lchild);
        EnQueue(Q, p->rchild);
    }

    while(!QueueEmpty(Q)) {
        DeQueue(Q, p);
        if(p != NULL) return 0;
    }
    return 1;
}
```

题型四：二叉排序树判断（算法设计）

真题（2019-2020）：判断一棵二叉树是否是二叉排序树

解题思路：

中序遍历BST得到有序序列
检查遍历结果是否严格递增

代码模板：

```
int pre = -32768;

int IsSearchTree(BSTree t) {
    if(t == NULL) return 1;

    int left = IsSearchTree(t->lchild);
    if(left == 0 || pre >= t->data)
        return 0;
    pre = t->data;

    return IsSearchTree(t->rchild);
}
```

题型五：Prim算法求最小生成树（应用题）

真题（2018-2019）：给定邻接矩阵，用Prim算法求最小生成树

解题步骤：

1. 从顶点v0开始
2. 找与已选集合相连的最小权边
3. 将该边加入生成树
4. 重复直到所有顶点加入
5. 累加所有权值

快速查询表

时间复杂度速查

情况	循环次数	复杂度
单层n循环	n次	O(n)
两层n循环	n²次	O(n²)
i×2递增循环	log₂n次	O(log n)
n×log n循环	nlog₂n次	O(nlog n)

稳定性速查

稳定排序	不稳定排序
直接插入	希尔排序
冒泡排序	快速排序
归并排序	简单选择
基数排序	堆排序

空间复杂度速查

算法	额外空间
堆排序	O(1)
快速排序	O(log n)（递归栈）
归并排序	O(n)

考场策略

选择题策略

- 概念题：**先排除明显错误的选项
- 复杂度题：**记住常见复杂度等级
- 链表题：**画图辅助理解
- 图论题：**注意有向/无向

应用题策略

- 二叉树遍历：**先画树再写序列
- 哈夫曼编码：**先建树再编码
- 哈希表：**按步计算每个地址
- 最小生成树：**列表追踪

算法题策略

- 先读懂题目：**明确输入输出
- 写结构体定义：**题目已给则跳过
- 分步实现：**每个功能单独写函数
- 注意边界：**空指针、循环终止条件

祝考试顺利！